背包

首先，我们将物品按照单位体积分类。

令 $dp_{i,j}$ 表示选了 i 种物品，是否存在总体积为 j 的方案。其中，第二维用 bitset 存储。

对于每一个 v ，我们将单位体积为 v 的物品加入到背包。

考虑加入k 种体积为 v 的物品可以构成哪些总体积。

容易得到以下结论：

- 总体积最小为 kv，设体积为 v 的数量最多的 k 种物品的个数总和为 s_k，那么总体积最大为 s_kv。
- 总体积为“首项为 kv，末项为 s_kv，公差为 v”的等差数列。

于是我们可以得到以下做法：

> 1. 枚举单位体积 v。
> 2. 枚举已经选的物品种类数 i。
> 3. 枚举当前总体积 j。
> 4. 枚举新增多少种单位体积为 v 的物品 k。
> 5. 枚举新增多少个单位体积为 v 的物品 c。
> 6. $dp'_{i+k,j+cv}| = dp_{i,j}$。

此时时间复杂度为 $O(n^2L^2)$。

考虑不枚举 j，在枚举完 k,c 之后，直接令 $dp'_{i+k}| = dp_i << cv$ 。

此时时间复杂度为 $O(n^2L^2/\omega)$。

考虑不枚举 c，由于之前提到的，如果把所有合法的 c 写出来，会得到等差数列。直接使用类似快速幂的方式用 dp_i 更新 dp'_{i+k}。

即令

$$f(x,y) = \begin{cases} bitset\ a\text{满足仅}a[0]=1 & (y=0) \\ f(x,y/2)|f(x,y/2) << (vy/2) & (y \neq 0, y \bmod 2 = 0) \\ f(x,y-1) << v|x & (y \bmod 2 = 1) \end{cases}$$

然后 $dp'_{i+k}| = f(dp_i, s_k - k + 1) << k$。

这样，我们就得到了一个 $O(nL(n+v)\log n/\omega)$ 的做法。

然后我们发现，我们可以在枚举完 i 之后，对于所有 $0 \le t \le \log L$，预处理出 $f(dp_i, 2^t)$，然后在更新时采用类似 ST 表的做法来得到 $f(dp_i, s_k - k + 1)$，即 $f(dp_i, x) = f(dp_i, 2^{\lfloor \log x \rfloor})|f(dp_i, 2^{\lfloor \log x \rfloor}) << ((x - 2^{\lfloor \log x \rfloor})v)$。

这样，我们就得到了一个 $O(n^2L/\omega + nLv\log L/\omega)$ 的做法。

考虑在处理 $f(dp_i, 2^t)$ 时，只对 $0 \le t \le \log \dfrac{L}{v}$ 作处理，仔细分析可以发现时间复杂度变成了 $O(n^2L/\omega + nLv/\omega)$，为了得到这个复杂度，我们只需证明 $O(\sum_{i=1}^{L} \lfloor \log \frac{L}{i} \rfloor) = O(L)$。

证明：

$$
\begin{aligned}
&\sum_{i=1}^{L}\left\lfloor \log\frac{L}{i}\right\rfloor \\
=&\sum_{k\geq 1}\sum_{i=1}^{L}\left[\left\lfloor \log\frac{L}{i}\right\rfloor \geq k\right] \\
=&\sum_{k\geq 1}\sum_{i=1}^{L}\left[\left\lfloor \frac{L}{i}\right\rfloor \geq 2^k\right] \\
=&\sum_{k\geq 1}\sum_{i=1}^{L}\left[L\geq 2^k i\right] \\
=&\sum_{k\geq 1}\sum_{i=1}^{L}\left[i\leq \left\lfloor \frac{L}{2^k}\right\rfloor\right] \\
=&\sum_{k\geq 1}\left\lfloor \frac{L}{2^k}\right\rfloor \\
\approx& L\sum_{k\geq 1}\frac{1}{2^k} \\
\approx& L
\end{aligned}
$$

参考代码

```
#include <bits/stdc++.h>
#define clr(x) memset(x,0,sizeof x)
#define For(i,a,b) for (int i=(a);i<=(b);i++)
#define Fod(i,b,a) for (int i=(b);i>=(a);i--)
#define fi first
#define se second
#define pb(x) push_back(x)
#define mp(x,y) make_pair(x,y)
#define outval(x) printf(#x" = %d\n",x)
#define outtag(x) puts("---------------"#x"---------------")
#define outarr(a,L,R) printf(#a"[%d..%d] = ",L,R);\
                        For(_x,L,R)printf("%d ",a[_x]);puts("")
using namespace std;
typedef long long LL;
LL read(){
    LL x=0,f=0;
    char ch=getchar();
    while (!isdigit(ch))
        f|=ch=='-',ch=getchar();
    while (isdigit(ch))
        x=(x<<1)+(x<<3)+(ch^48),ch=getchar();
    return f?-x:x;
}
const int N=2005;
int n,m,L;
int s[N];
vector <int> c[N];
bitset <N> dp[N],st[15],ans;
int pre[N];
int Log[N];
int main(){
    n=read(),L=read();
    For(i,2,N-1)
        Log[i]=Log[i>>1]+1;
    For(i,1,n){
```

```
        int C=read(),v=read();
        c[v].pb(C);
    }
    m=read();
    For(i,1,m)
        s[i]=read();
    For(i,0,n)
        dp[i].reset();
    dp[0][0]=1;
    For(v,1,2000){
        if (c[v].empty())
            continue;
        sort(c[v].begin(),c[v].end());
        reverse(c[v].begin(),c[v].end());
        int len=c[v].size();
        pre[0]=0;
        For(i,0,len-1)
            pre[i+1]=pre[i]+c[v][i];
        Fod(i,n-1,0){
            st[0]=dp[i];
            For(j,1,10)
                if ((v<<j)<=L)
                    st[j]=st[j-1]<<(v<<(j-1))|st[j-1];
                else
                    break;
            For(j,1,len){
                if (i+j>n)
                    break;
                int now=min(pre[j]-j+1,L/v),d=Log[now];
                dp[i+j]|=(st[d]<<(v*(now-(1<<d)))|st[d])<<(v*j);
            }
        }
    }
    ans.reset();
    For(i,1,m)
        ans|=dp[s[i]];
    For(i,0,L)
        if (ans[i])
            printf("%d ",i);
    puts("");
    return 0;
}
```